

Chapter 1

Conditionals

In this chapter, we'll use functions and a new feature—conditional statements—to search for Pythagorean triples. A Pythagorean triple is a set of integers, like 3, 4, and 5, that are the lengths of the sides of a right triangle. Mathematically, it's a set of integers a , b , and c such that $a^2 + b^2 = c^2$. This example will also demonstrate the *incremental development* process we talked about in Chapter 3.

1.1 Relational Operators

Suppose we have three variables, `a`, `b`, and `c`, and we want to check whether they form a Pythagorean triple. We can use the equality operator (`==`) to compare two values:

```
>> a = 3;
>> b = 4;
>> c = 5;
>> a^2 + b^2 == c^2

ans = logical 1
```

The result is a *logical* data type, which means it's either 1, which means “true,” or 0, which means “false.” In fact, you can use `true` as substitute for 1 and `false` as shorthand for 0:

```
>> true
ans =
    logical
     1
>> true == 1
ans =
    logical
     1
```

Here's an example where the result is false:

```
>> c = 6;  
>> a^2 + b^2 == c^2  
ans = logical 0
```

It's a common error to use the assignment operator (=) instead of the equality operator (==). If you do, you get an error:

```
>> a^2 + b^2 = c^2  
a^2 + b^2 = c^2  
|  
Error: Incorrect use of '=' operator.  
To assign a value to a variable, use '='.  
To compare values for equality, use '=='.
```

The equality operator is one of several *relational operators*, so called because they test relations between values. For example, $x < 10$ is true (1) if the value of x is less than 10 or false (0) if otherwise. And $x > 0$ is true if x is greater than 0.

The other relational operators are $\leq$ for “less or equal,” $\geq$ for “greater or equal,” and $\sim$ for “not equal.”

1.2 if Statement

Now suppose that when we find a Pythagorean triple we want to display a message. The `if` statement allows you to check for certain conditions and execute statements if the conditions are met. For example:

```
if a^2 + b^2 == c^2  
    disp("Yes, that is a Pythagorean triple.")  
end
```

The syntax is similar to a `for` loop. The first line specifies the condition we're interested in. If the condition is true, MATLAB executes the *body* of the statement, which is the indented sequence of statements between the `if` and the `end`.

MATLAB doesn't require you to indent the body of an `if` statement, but it makes your code more readable, so you should do it.

If the condition is not satisfied, the statements in the body are not executed.

Sometimes there are alternative statements to execute when the condition is false. In that case, you can extend the `if` statement with an `else` clause.

The complete version of the previous example might look like this:

```
if a^2 + b^2 == c^2  
    disp("Yes, that is a Pythagorean triple.")  
else  
    disp("No, that is not a Pythagorean triple.")  
end
```

Statements like `if` and `for` that contain other statements are called *compound* statements. All compound statements finish with `end`.

1.3 Incremental Development

Now that we have relational operators and `if` statements, let's start writing the program.

Here are the steps we will follow to develop the program incrementally:

1. Write a script named *find_triples.m* and start with a loop that enumerates values of `a` and displays them.
2. Write a second loop that enumerates values of `b` and a third loop that enumerates values of `c`.
3. Use the `if` statement from the previous section to check whether `a`, `b`, and `c` form a Pythagorean triple.
4. Display the values that pass the test.
5. Transform the script into a function and make it take an input variable that specifies the range to search.

Along the way, we'll optimize the program to eliminate unnecessary work.

1.4 Logical Functions

The first step is to create a *logical function*, which is a function that returns a logical value. The following function takes three input variables, `a`, `b`, and `c`, and returns true (1) if they form a Pythagorean triple and false (0) otherwise.

```
function res = is_pythagorean(a, b, c)
    if a^2 + b^2 == c^2
        res = 1;
    else
        res = 0;
    end
end
```

We can use this function like so:

```
>> is_pythagorean(3, 4, 5)
ans = 1
```

But we can write the same function more concisely, like this:

```
function res = is_pythagorean(a, b, c)
    res = a^2 + b^2 == c^2;
end
```

The result of the equality operator is a logical value, which we can assign directly to `res`.

Put this function in a file called *is_pythagorean.m*, so we can use it as part of our program.

1.5 Nested Loops

The next step is to write loops that enumerate different values of `a`, `b`, and `c`. Create a new file called *find_triples.m* where we'll develop the rest of the program.

We'll start with a loop for `a`:

```
for a=1:3
    a
end
```

It might seem silly to start with such a simple program, but this is an essential element of incremental development: start simple and test as you go.

The output is as expected.

```
1
2
3
```

Now we'll add a second loop for `b`. It might be tempting to write something like this:

```
for a=1:3
    disp(a)
end
for b=1:4
    disp(b)
end
```

But that loops through the values of `a` and then loops through the values of `b`, and that's not what we want.

Instead, we want to consider every possible pair of values, like this:

```
for a=1:3
    for b=1:4
        disp([a,b])
    end
end
```

Now one loop is inside the other. The inner loop gets executed three times, once for each value of `a`, so here's what the output looks like (I've adjusted the spacing to make the structure clear):

```
>> find_triples
      1      1
```

1	2
1	3
1	4
2	1
2	2
2	3
2	4
3	1
3	2
3	3
3	4

The left column shows the values of `a` and the right column shows the values of `b`.

The next step is to search for values of `c` that might make a Pythagorean triple. The largest possible value for `c` is `a + b`, because otherwise we couldn't form a triangle (see <https://greenteapress.com/matlab/triangle>).

```
for a=1:3
    for b=1:4
        for c=1:a+b
            disp([a,b,c])
        end
    end
end
```

After each small change, run the program again and check the output.

1.6 Putting It Together

Now instead of displaying all of the triples, we'll add an `if` statement and display only Pythagorean triples:

```
for a=1:3
    for b=1:4
        for c=1:a+b
            if is_pythagorean(a, b, c)
                disp([a,b,c])
            end
        end
    end
end
```

The result is just one triple:

```
>> find_triples
    3    4    5
```

You might notice that we're wasting some effort here. After checking the case when `a` is 1 and `b` is 2, there's no point in checking the case when `a` is 2 and `b` is 1. We can save the extra work by adjusting the range of `b`:

```
for b=a:4
```

We can save even more work by adjusting the range of `c`:

```
for c=b:a+b
```

Here's the final version:

```
for a=1:3
    for b=a:4
        for c=b:a+b
            if is_pythagorean(a, b, c)
                disp([a,b,c])
            end
        end
    end
end
```

1.7 Encapsulation and Generalization

As a script, this program has the side effect of assigning values to `a`, `b`, and `c`, which would be bad if any of those names were in use. By wrapping the code in a function, we can avoid name collisions; this process is called *encapsulation* because it isolates this program from the workspace.

The first draft of the function takes no input variables:

```
function res = find_triples()
    for a=1:3
        for b=a:4
            for c=b:a+b
                if is_pythagorean(a, b, c)
                    disp([a,b,c])
                end
            end
        end
    end
end
```

The empty parentheses in the signature are not necessary, but they make it apparent that there are no input variables. Similarly, it's a good idea when calling the new function to use parentheses as a reminder that it's a function, not a script:

```
>> find_triples()
```

The output variable isn't necessary, either; it never gets assigned a value. But I put it there as a matter of habit and so my function signatures all have the same structure.

The next step is to generalize this function by adding input variables. The natural generalization is to replace the constant values 3 and 4 with a variable, so we can search an arbitrarily large range of values.

```
function res = find_triples(n)
    for a=1:n
        for b=a:n
            for c=b:a+b
                if is_pythagorean(a, b, c)
                    disp([a,b,c])
                end
            end
        end
    end
end
```

Here are the results for the range from 1 to 15:

```
>> find_triples(15)
     3     4     5
     5    12    13
     6     8    10
     8    15    17
     9    12    15
```

The triples 5, 12, 13 and 8, 15, 17 are new, but the others are just multiples of the 3, 4, 5 triangle.

1.8 Adding a continue Statement

As a final improvement, let's modify the function so it only displays the "lowest" of each Pythagorean triple, and not the multiples.

The simplest way to eliminate the multiples is to check whether **a** and **b** share a common factor. If they do, dividing both by the common factor yields a smaller, similar triangle that has already been checked.

MATLAB provides a `gcd` function that computes the greatest common divisor of two numbers. If `gcd(a,b)` is greater than 1, **a** and **b** share a common factor, and we can use the `continue` statement to skip to the next pair. Listing 1.1 contains the final version of this function:

Listing 1.1: Our final Pythagorean triples function

```
function res = find_triples(n)
    for a=1:n
```

```
    for b=a:n
        for c=b:a+b
            if gcd(a,b) > 1
                continue
            end
            if is_pythagorean(a, b, c)
                disp([a,b,c])
            end
        end
    end
end
end
```

The `continue` statement causes the program to end the current iteration immediately, jump to the top of the loop, and “continue” with the next iteration.

In this case, since there are three loops, it might not be obvious which loop to jump to, but the rule is to jump to the innermost loop (which is what we want).

Here are the results with `n = 40`:

```
>> find_triples(40)
    3     4     5
    5    12    13
    7    24    25
    8    15    17
    9    40    41
   12    35    37
   20    21    29
```

1.9 How Functions Work

Let’s review the sequence of steps that occur when you call a function:

1. Before the function starts running, MATLAB creates a new workspace for it.
2. MATLAB evaluates each of the arguments and assigns the resulting values, in order, to the input variables (which live in the *new* workspace).
3. The body of the code executes. Somewhere in the body a value gets assigned to the output variable(s).
4. The function’s workspace is destroyed; the only thing that remains is the value of the output variable(s) and any side effects the function had (like displaying values).
5. The program resumes from where it left off. The value(s) of the function call is the value(s) of the output variable(s).

When you're reading a program, and you come to a function call, there are two ways to interpret it. You can think about the mechanism I just described, and follow the execution of the program into the function and back, or you can assume that the function works correctly, and go on to the next statement after the function call.

When you use a built-in function, it's natural to assume that it works, in part because you don't usually have access to the code in the body of the function. But when you start writing your own functions, you might find yourself following the "flow of execution." This can be useful while you are learning, but as you gain experience, you should get more comfortable with the idea of writing a function, testing it to make sure it works, and then forgetting about the details of how it works.

Forgetting about details is called *abstraction*; in the context of functions, abstraction means forgetting about *how* a function works and just assuming (after appropriate testing) that it works. For many people, it takes some time to get comfortable with functions. If you are one of them, you might be tempted to avoid functions, and sometimes you can get by without them.

But experienced programmers use functions extensively, for several good reasons. First, each function has its own workspace, so using functions helps avoid name collisions. Functions also lend themselves to incremental development: you can debug the body of the function first (as a script), then encapsulate it as a function, and then generalize it by adding input variables.

Also, functions allow you to divide a large problem into small pieces, work on the pieces one at a time, and then assemble a complete solution.

Once you have a function working, you can forget about the details of how it works and concentrate on what it does. This process of abstraction is an important tool for managing the complexity of large programs.

1.10 Chapter Review

In this chapter, we encountered relational operators and `if` statements, and we used them to develop a program that searches for Pythagorean triples. We wrote a *logical function*, which is a function that returns a logical value (1 for "true" or 0 for "false").

We also saw an example of *incremental development*, or developing programs gradually, adding just a few lines of code at a time and testing as you go. If you develop programs this way, you will have fewer bugs, and you will find them more quickly.

This chapter defined two new terms: *encapsulation* is the process of wrapping part of a program in a function in order to limit interactions (including name collisions) between the function and the rest of the program; *abstraction* is the process of ignoring the details of how a function works in order to focus on a simpler model of what the function does.

The next chapter introduces a new tool, called `fzero`, that we'll use to solve nonlinear equations.

1.11 Exercise

Before you go on, you might want to work on the following exercise.

Exercise 1.1. There is an interesting connection between Fibonacci numbers and Pythagorean triples. If F is a Fibonacci sequence, then

$$(F_i F_{i+3}, 2F_{i+1} F_{i+2}, F_{i+1}^2 + F_{i+2}^2)$$

is a Pythagorean triple, for all $i \geq 1$.

Write a function named `fib_triple` that takes `n` as an input variable, computes the first `n` Fibonacci numbers, stores them in a vector, and checks whether this formula produces Pythagorean triples for numbers in the sequence.